



LockMySeat: Simulação de Sistema Distribuído para Reserva de Assentos em Cinema

Débora Almeida Buriti da Silva | Graduada em Análise e Desenvolvimento de Sistemas, Cesar School, Embarque Digital;

dabs2@cesar.school

Mirella Emily Bezerra Santana | Graduada em Análise e Desenvolvimento de Sistemas, Cesar School, Embarque Digital;

mebs@cesar.school

Myllena Cristiane Ribeiro Navarro Lins | Graduada em Análise e Desenvolvimento de Sistemas, Cesar School, Embarque Digital;

mcrnl@cesar.school

Resumo

O presente artigo apresenta o desenvolvimento do LockMySeat, uma aplicação distribuída baseada no modelo Cliente-Servidor, cujo objetivo é simular um sistema de reserva de assentos em cinema com suporte à concorrência e paralelismo. O projeto foi desenvolvido na linguagem Python, utilizando threads e mecanismos de sincronização (lock) para garantir a integridade dos dados durante o acesso simultâneo de múltiplos clientes. Por meio da comunicação via TCP Socket, o servidor é capaz de atender diferentes requisições em paralelo, assegurando a consistência das operações de reserva e consulta. Os resultados obtidos demonstram o correto funcionamento da aplicação e evidenciam a importância da sincronização em sistemas distribuídos. Além disso, o trabalho possibilitou à equipe reforçar os conceitos teóricos e colocar em prática os conteúdos estudados na disciplina.

Palavras-chave: Sistemas Distribuídos; Concorrência; Threads; Cliente-Servidor.

1. Introdução

O presente projeto é derivado da disciplina Fundamentos de Computação Concorrente, Paralela e Distribuída, cujo objetivo geral é compreender os fundamentos do desenvolvimento de aplicações escaláveis, tolerantes a falhas e de alta disponibilidade. Entre os objetivos específicos da disciplina estão: entender os principais conceitos relacionados à concorrência e ao paralelismo, aprender as características de sistemas distribuídos, conhecer a arquitetura dos principais middlewares e dominar as boas práticas associadas ao projeto de sistemas distribuídos. Esses conhecimentos fornecem a base teórica e prática necessária para o desenvolvimento de soluções modernas que demandam processamento simultâneo e comunicação eficiente entre múltiplas entidades.

A partir desse contexto, foi proposto o Projeto 1, que engloba os conteúdos abordados na primeira unidade da disciplina. O objetivo do projeto é desenvolver uma solução prática que utilize arquitetura distribuída, aplicando os conceitos estudados em sala de aula sobre concorrência, paralelismo e sistemas distribuídos. Para isso, cada grupo deveria implementar uma aplicação funcional, capaz de ser testada e reproduzida, integrando conceitos como threads, processos, sincronização e comunicação entre processos ou threads.

A arquitetura escolhida para o desenvolvimento do projeto foi o modelo Cliente-Servidor, amplamente utilizado em sistemas distribuídos. Nessa abordagem, o servidor é responsável por gerenciar os recursos, processar as requisições e manter a integridade dos dados, enquanto o cliente atua como o ponto de interação do usuário, enviando solicitações e recebendo respostas. Esse tipo de arquitetura permite a comunicação simultânea de múltiplos clientes com um único servidor, sendo ideal para explorar conceitos como controle de concorrência, sincronização de threads e comunicação via sockets — fundamentos essenciais da disciplina.

Com base nesses conceitos, surgiu o LockMySeat, uma aplicação voltada para a reserva de assentos em cinemas, desenvolvida a partir do interesse em comum do grupo pelo universo cinematográfico e pela vontade de resolver, de forma prática e didática, o problema de conflitos em reservas simultâneas. O sistema foi projetado para simular o ambiente de um cinema, permitindo que múltiplos usuários acessem o servidor ao mesmo tempo, visualizem sessões disponíveis e realizem reservas sem que ocorram inconsistências nos dados. A solução técnica e os resultados obtidos serão detalhados nas próximas seções deste artigo.

2. Metodologia

O desenvolvimento do projeto baseou-se em diferentes fontes de estudo e apoio técnico, dentre elas: documentos e materiais da disciplina, exemplos de código e exercícios disponibilizados pelo professor, fontes complementares de pesquisa (como fóruns e tutoriais disponíveis na internet), além do uso de Agentes de Inteligência Artificial, empregados como ferramenta de apoio para revisão de código.

O segundo passo foi a definição da arquitetura e das tecnologias a serem utilizadas na aplicação. A arquitetura escolhida foi o modelo Cliente-Servidor, por se adequar à proposta solicitada pelo professor e aos objetivos definidos para o projeto. O servidor concentra a lógica principal da aplicação, gerenciando os recursos compartilhados (como o mapa de assentos), enquanto os clientes representam os usuários que enviam requisições para listar sessões, visualizar assentos e realizar reservas. Para evitar condições de corrida, foi utilizada

uma estrutura de bloqueio (lock), que garante que apenas uma thread por vez possa modificar os dados de reserva. Assim, o sistema evita a duplicidade de reservas para um mesmo assento.

Escolhemos a linguagem Python 3, pela sua clareza e pelas bibliotecas nativas que oferecem suporte direto à programação concorrente e comunicação em rede. Dentre essas bibliotecas, utilizamos *socket* para realizar comunicação TCP entre cliente e servidor. *Threading* para a criação de múltiplas threads, permitindo o atendimento simultâneo de vários clientes. *Json* para armazenamento e manipulação dos dados das sessões e assentos. *Time* e *datetime* utilizadas em logs e simulações de processamento.

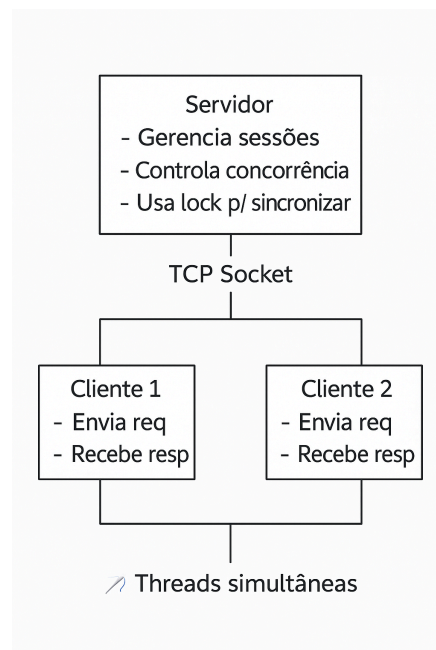


Figura 1: Diagrama da arquitetura. Fonte: Autoras, 2025.

O Diagrama 1 ilustra a arquitetura do sistema LockMySeat, destacando a interação entre os componentes servidor e clientes.

3. Resultados

O LockMySeat apresentou resultados favoráveis, uma vez que demonstrou o funcionamento esperado de um sistema distribuído com controle de concorrência e sincronização de threads. O servidor foi capaz de gerenciar múltiplas conexões simultâneas via TCP Socket, atendendo diversos clientes em paralelo e garantindo a consistência dos dados de reserva. Durante os testes, o servidor exibiu corretamente a criação de uma nova thread para cada cliente conectado, evidenciando o paralelismo no atendimento das requisições. Cada cliente pôde realizar as atividades de forma independente, sem impactar o desempenho dos demais usuários.

```
(venv) enio@myllena-dcm4a-4:~/Downloads/lockmyseat-dev$ python3 sever.py
Servidor de Cinema iniciado em localhost:5555
21:13:40
=====
✓ Cliente #1 conectado: ('127.0.0.1', 49100)
🧵 Thread iniciada para Cliente #1
📁 Cliente #1: LISTAR_SESSOES
✗ Cliente #1 desconectado: ('127.0.0.1', 49100)
✓ Cliente #2 conectado: ('127.0.0.1', 39334)
🧵 Thread iniciada para Cliente #2
✓ Cliente #3 conectado: ('127.0.0.1', 52928)
🧵 Thread iniciada para Cliente #3
✓ Cliente #4 conectado: ('127.0.0.1', 52932)
🧵 Thread iniciada para Cliente #4
📁 Cliente #4: RESERVAR 1 A1
✓ Cliente #4 reservou: Sessão 1, Assento A1
✗ Cliente #4 desconectado: ('127.0.0.1', 52932)
📁 Cliente #3: RESERVAR 1 A1
✗ Cliente #3 tentou reservar A1 (OCUPADO)
✗ Cliente #3 desconectado: ('127.0.0.1', 52932)
```

Figura 2: Servidor ativo e em execução. Fonte: Autoras, 2025.

Para validar os fluxos desenvolvidos, executamos alguns testes, como teste de condição de corrida, teste com múltiplos clientes e consultas simultâneas.

Teste de Condição de Corrida

```
enio@myllena-dcm4a-4:~/Downloads/lockmyseat-dev$ python3 test_concurrency.py
TESTES DE CONCORRÊNCIA - SISTEMA DE CINEMA
=====
Certifique-se de que o servidor está rodando!
=====
Pressione ENTER para iniciar os testes...

=====
🔧 TESTE DE CONDIÇÃO DE CORRIDA
=====
Dois clientes tentarão reservar o mesmo assento (A1)
Apenas um deve conseguir!

🟦 Cliente 2 conectado
🟦 Cliente 1 conectado
📁 Cliente 1 enviando: RESERVAR 1 A1

=====
📁 RESPOSTA para Cliente 1:
OK: Assento A1 reservado com sucesso!
Filme: Oppenheimer
Horário: 19:00
Sala: A1
=====
📁 Cliente 2 enviando: RESERVAR 1 A1
```

```

Sala: A1
=====
👤 Cliente 2 enviando: RESERVAR 1 A1
=====
👤 RESPOSTA para Cliente 2:
INDISPONIVEL: Assento A1 já está ocupado
=====
✅ Teste concluído!
  
```

Figuras 3 e 4: Teste de Condição de Corrida. Fonte: Autoras, 2025.

Dois clientes tentaram reservar o mesmo assento (A1) na mesma sessão, quase simultaneamente. Na ocasião, o mecanismo de bloqueio (lock) impediu o acontecimento, fazendo com que apenas o primeiro cliente obtivesse sucesso na reserva. Na figura abaixo, pode-se observar o fluxo percorrido no processo de reserva.

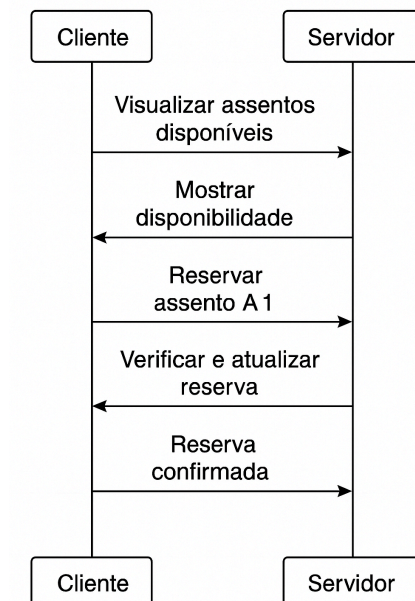


Figura 5: Fluxo de comunicação entre cliente e servidor no processo de reserva. Fonte: Autoras, 2025.

Teste com Múltiplos Usuários

```

=====
🔧 TESTE COM MÚLTIPLOS CLIENTES
=====
5 clientes reservando assentos diferentes simultaneamente

🟢 Cliente 2 conectado
🟢 Cliente 1 conectado
🟢 Cliente 3 conectado
🟢 Cliente 4 conectado
🟢 Cliente 5 conectado
👤 Cliente 1 enviando: RESERVAR 1 A1
=====
👤 RESPOSTA para Cliente 1:
INDISPONIVEL: Assento A1 já está ocupado
=====
  
```

```
👤 Cliente 2 enviando: RESERVAR 1 A2
=====
👤 RESPOSTA para Cliente 2:
OK: Assento A2 reservado com sucesso!
Filme: Oppenheimer
Horário: 19:00
Sala: A1
=====

👤 Cliente 3 enviando: RESERVAR 1 B1
=====
👤 RESPOSTA para Cliente 3:
OK: Assento B1 reservado com sucesso!
Filme: Oppenheimer
Horário: 19:00
Sala: A1
=====

👤 Cliente 4 enviando: RESERVAR 1 B2
=====
👤 RESPOSTA para Cliente 4:
OK: Assento B2 reservado com sucesso!
Filme: Oppenheimer
Horário: 19:00
Sala: A1
=====

👤 Cliente 5 enviando: RESERVAR 1 C1
=====
👤 RESPOSTA para Cliente 5:
OK: Assento C1 reservado com sucesso!
Filme: Oppenheimer
Horário: 19:00
Sala: A1
=====

✅ Teste concluído!
```

Figuras 6, 7 e 8: Teste com Múltiplos Usuários. Fonte: Autoras, 2025.

Cinco clientes diferentes executaram requisições de reserva para assentos distintos, de forma simultânea. Todas as operações foram concluídas com êxito, sem travamentos ou inconsistências.

Teste de Consulta Simultânea

```
=====
🔧 TESTE DE CONSULTAS SIMULTÂNEAS
=====
3 clientes consultando assentos simultaneamente

👤 Cliente 3 consultando sessão 1
👤 Cliente 1 consultando sessão 1
👤 Cliente 2 consultando sessão 1
✅ Cliente 2 recebeu resposta
✅ Cliente 3 recebeu resposta
✅ Cliente 1 recebeu resposta

✅ Teste concluído!

=====
🎉 TODOS OS TESTES CONCLUÍDOS!
=====
enio@myllena-dcm4a-4: ~/Downloads/lockmyseat-dev$
```

Figura 9: Teste de Consulta Simultânea. Fonte: Autoras, 2025.

Três clientes realizaram, ao mesmo tempo, a consulta de assentos disponíveis em uma mesma sessão. Novamente, não foram localizados erros ou inconsistências. O desempenho geral da aplicação mostrou-se consistente em todos os cenários simulados.

4. Conclusões

O desenvolvimento do LockMySeat possibilitou a aplicação prática dos principais conceitos abordados em sala de aula, mostrando como teoria e prática se conectam em código funcional. Desta forma, compreendemos de forma mais profunda os desafios e decisões envolvidas na construção de aplicações que exigem controle de acesso simultâneo e comunicação entre processos.

Além do aprendizado técnico, o projeto proporcionou insights valiosos para a equipe, especialmente em relação à integração de conhecimentos, versionamento de código, colaboração e gestão de tempo. Conciliar o desenvolvimento do projeto com outras demandas acadêmicas, profissionais e pessoais exigiu priorização de atividades e comprometimento coletivo, tornando o processo não apenas um exercício de programação, mas também uma experiência de amadurecimento profissional.

Como possibilidades de evolução, destacamos primeiramente a criação de uma interface gráfica para o cliente. Esse aprimoramento tornaria a interação com o sistema mais intuitiva e amigável, permitindo que o usuário realize as atividades sem depender apenas do terminal. Além disso, identificamos como evolução a implementação de persistência dos dados em banco de dados, pois atualmente as informações são armazenadas apenas em memória durante a execução. Com um banco de dados, seria possível manter o histórico de sessões e reservas, garantindo maior durabilidade e escalabilidade à aplicação.

Por fim, concluímos que o LockMySeat atingiu os objetivos propostos pelo Projeto 1, oferecendo uma solução funcional, alinhada com o funcionamento de um sistema distribuído.